

Ingeniería del Software de Componentes y Sistemas Distribuidos

Práctica 1

Presentación

Esta Práctica profundiza en el desarrollo de software basado en componentes e introduce la plataforma Java EE. La Práctica 1 adapta el diseño de la PEC 2 para su implementación en una plataforma Java EE. Incluye también la instalación de todo el software necesario para la codificación final del sistema. Se recomienda haber estudiado el material del Laboratorio e iniciado el estudio de los módulos 4, 5 y 6.

Competencias

En esta Práctica se trabaja la siguiente competencia del Grado en Ingeniería Informática:

- Saber construir aplicaciones informáticas mediante técnicas de desarrollo, integración y reutilización.

Objetivos

Los objetivos concretos de esta Práctica son:

- Conocer la programación orientada a componentes como técnica de implementación de las arquitecturas de software.
- Ver cómo se materializa (y adapta) este marco teórico sobre las plataformas tecnológicas actuales, con especial atención a Java EE.

Descripción de la Práctica a realizar

La práctica 1 la podemos dividir en tres partes claramente diferenciadas que se corresponden a los ejercicios 1, 2 y 3.

- En el primer ejercicio se debe hacer el refinamiento y diseño con Java EE de los componentes que formaban parte de la aplicación MyHealth identificados en la PEC2.
- En el segundo ejercicio se realizará una instalación y configuración del software que servirá para realizar la práctica 2.
- En el tercer ejercicio se proporcionará una parte de la aplicación de la tienda de animales ya desarrollada y se pedirá una pequeña modificación, también de cara a aprender y preparar la práctica 2.

A continuación, os presentamos los enunciados de estos ejercicios:

Ejercicio 1 (7 puntos)

En la PEC2 se hizo un diagrama de casos de uso y la especificación del punto de vista de la computación con los conjuntos de componentes arquitectónicos en los que se podía dividir el sistema **MyHealth**. En este primer ejercicio de la práctica 1 se pedirán dos cosas:

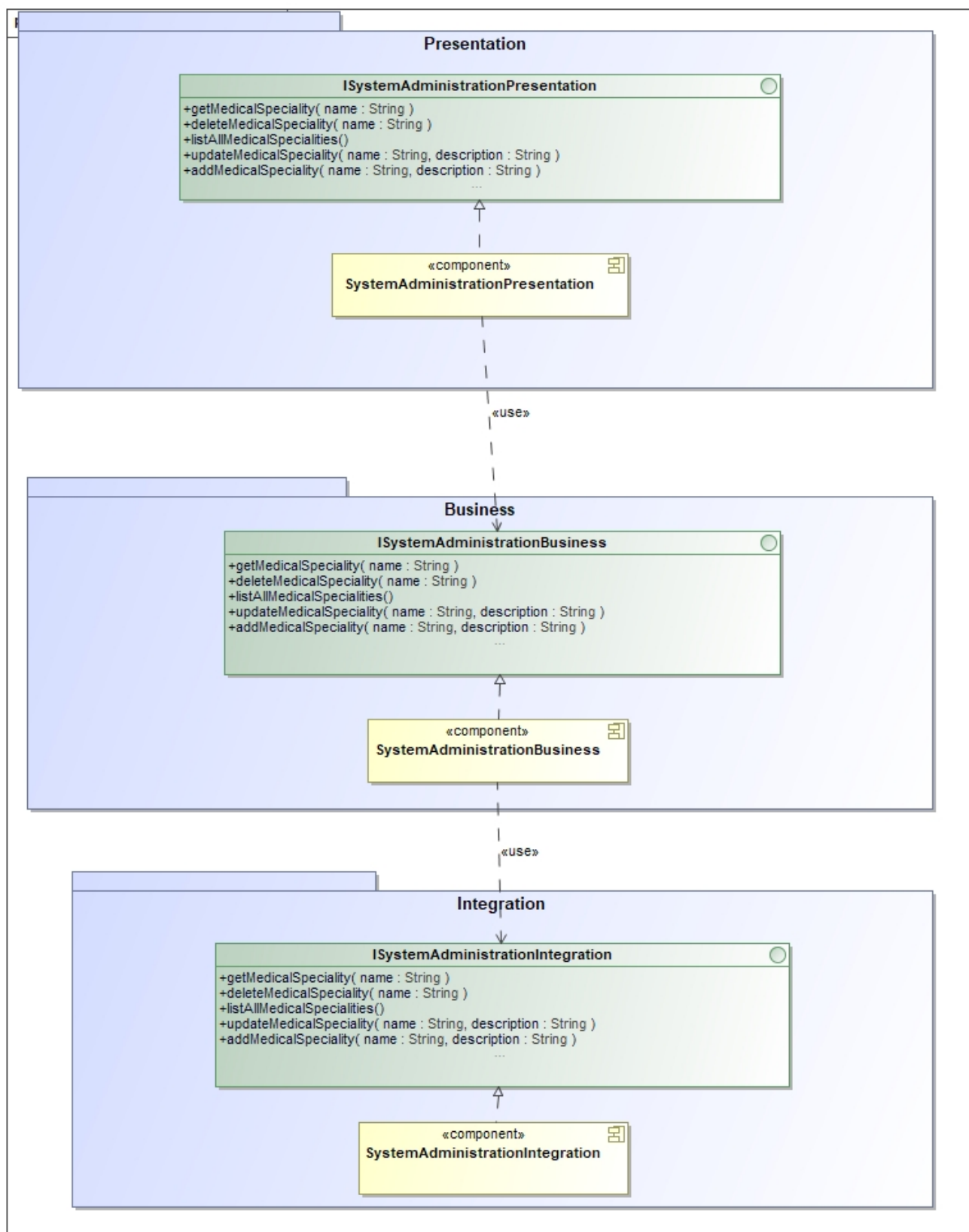
Apartado a)

Refinar un componente del punto de vista de la computación obtenido en la PEC2 para obtener un diagrama de componentes software. El sistema que tenéis que refinar debe ofrecer las siguientes funcionalidades:

- Consultar todos los tipos de especialidad médica
- Añadir un tipo de especialidad médica
- Borrar un tipo de especialidad médica
- Actualizar la información de un tipo de información médica

Para simplificar el ejercicio, **NO** hace falta ninguna gestión de usuarios ni que implementéis la parte correspondiente a la identificación y autorización. Considerad que se trata de una funcionalidad pública.

El diagrama de partida que se debe refinar es el correspondiente al componente ISystemAdministration del punto de vista de la computación entregado como solución para la PEC2



Tened en cuenta que en la PEC2 ya se tomó un conjunto de decisiones que llevaba inherente el estilo arquitectónico propuesto y que el cliente ha aceptado:

- Los usuarios deben acceder a la aplicación sin instalar ningún software especial, únicamente un navegador.
- La aplicación tendrá una arquitectura por capas.
- Tendremos tres capas: capa de presentación, capa de negocio y capa de integración (también llamada de administración de datos).
- Implementaremos las tres capas haciendo uso de una tecnología de componentes distribuida.

Qué hay que hacer:

- Refinar el subsistema hasta conseguir un conjunto de componentes software con las operaciones y las premisas descritas en el enunciado. Esto implica refinar todos los componentes de cada una de las capas: presentación, lógica de negocio e integración.
- Razonar todas las decisiones de diseño que toméis para cada una de las capas del componente.
- Si echáis en falta algún método en el diagrama de componentes del subsistema de partida que se debe refinar, se puede agregar. Comentarlo al principio del informe.

Qué hay que entregar:

- Todos los diagramas de componentes refinados que se van obteniendo en el proceso de refinamiento hasta llegar al diagrama de componentes final.
- Para cada paso de refinamiento es necesario el razonamiento de todas las decisiones de diseño.

Apartado b)

Aplicando las decisiones de diseño que se consideren oportunas, teniendo en cuenta que la tecnología de implementación será Java EE, realizad el diseño aplicando el perfil Java EE que se ha visto en el módulo 6. Aparte de las consideraciones anteriores, también se puede suponer lo siguiente:

- Utilizar Java EE como tecnología de componentes distribuida.
- Los datos de los clientes se almacenarán en una base de datos relacional.
- El acceso a la capa de negocio puede ser tanto remoto como local.
- El acceso a la capa de integración (persistencia) sólo será local.

Qué hay que hacer:

- Diseñar los componentes software obtenidos en el apartado a) teniendo en cuenta que la tecnología de implementación será Java EE.
- Razonar todas las decisiones de diseño.

Qué hay que entregar:

- Todos los diagramas de componentes refinados con el perfil Java EE aplicado.
- Razonar todas las decisiones de diseño que tenga que tomar para aplicar el perfil Java EE

NOTAS:

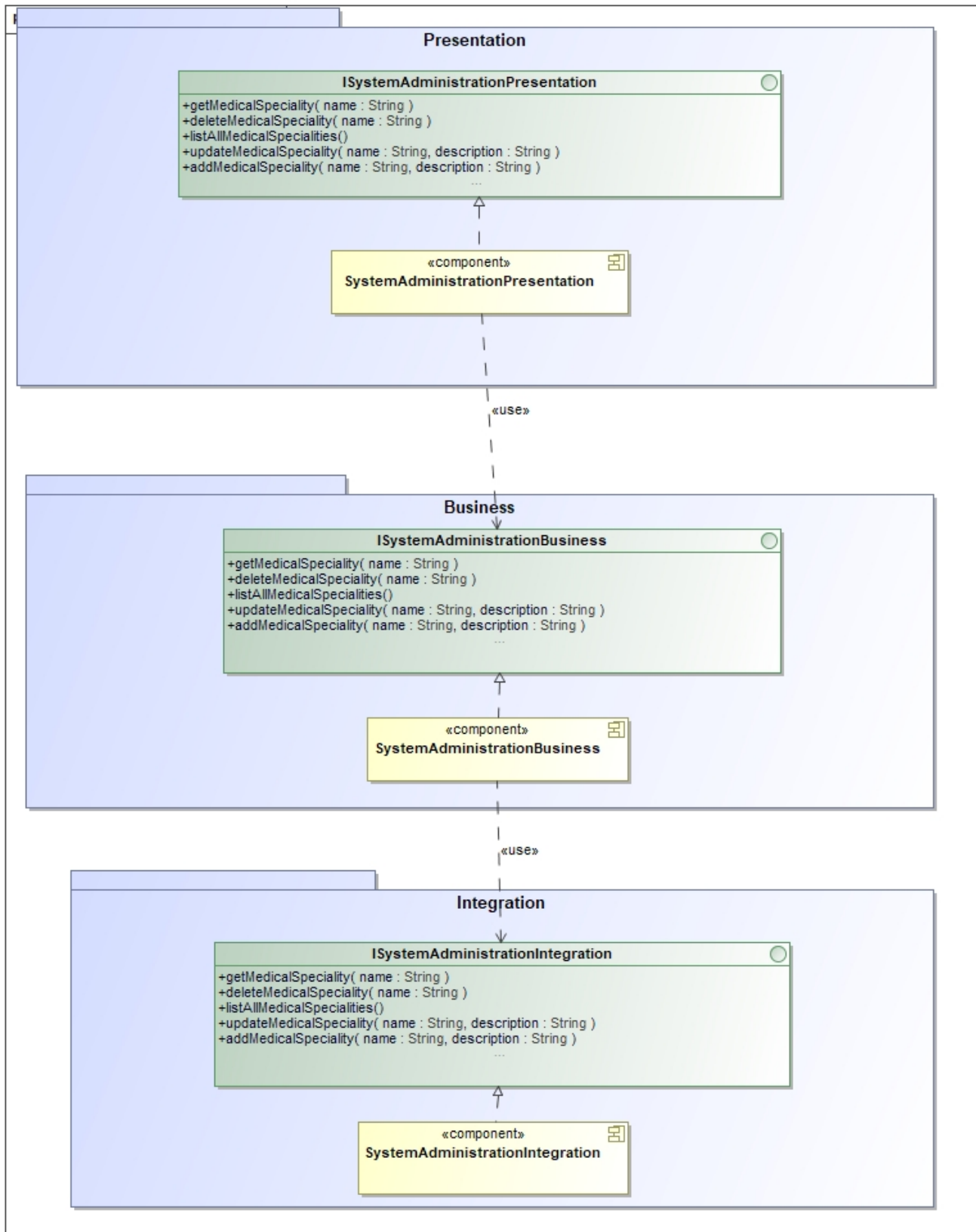
Tomad como referencia el proceso de diseño del componente **OperacionesBasicas** que encontraréis en el módulo 6 de los materiales y el diseño del caso práctico. Tened también en cuenta que, si bien el proceso que deberá seguir será similar, las decisiones de diseño que se tomen no tienen por qué coincidir con las que se toman en el módulo 6.

Solución

Ejercicio 1 apartados a) i b)

Partimos del diagrama de componentes que representaba el punto de vista de la computación de **MyHealth**. En esta práctica 1 os pedimos el refinamiento y diseño con Java EE de una parte de este sistema, concretamente el componente que se encarga de **gestionar los tipos de especialidades médicas** que hay en el sistema.

Partiremos pues de este diagrama:



El componente lo hemos dividido en tres capas tal como nos indicaban las premisas de la PEC2. Para hacer el diseño de los componentes de cada capa haremos:

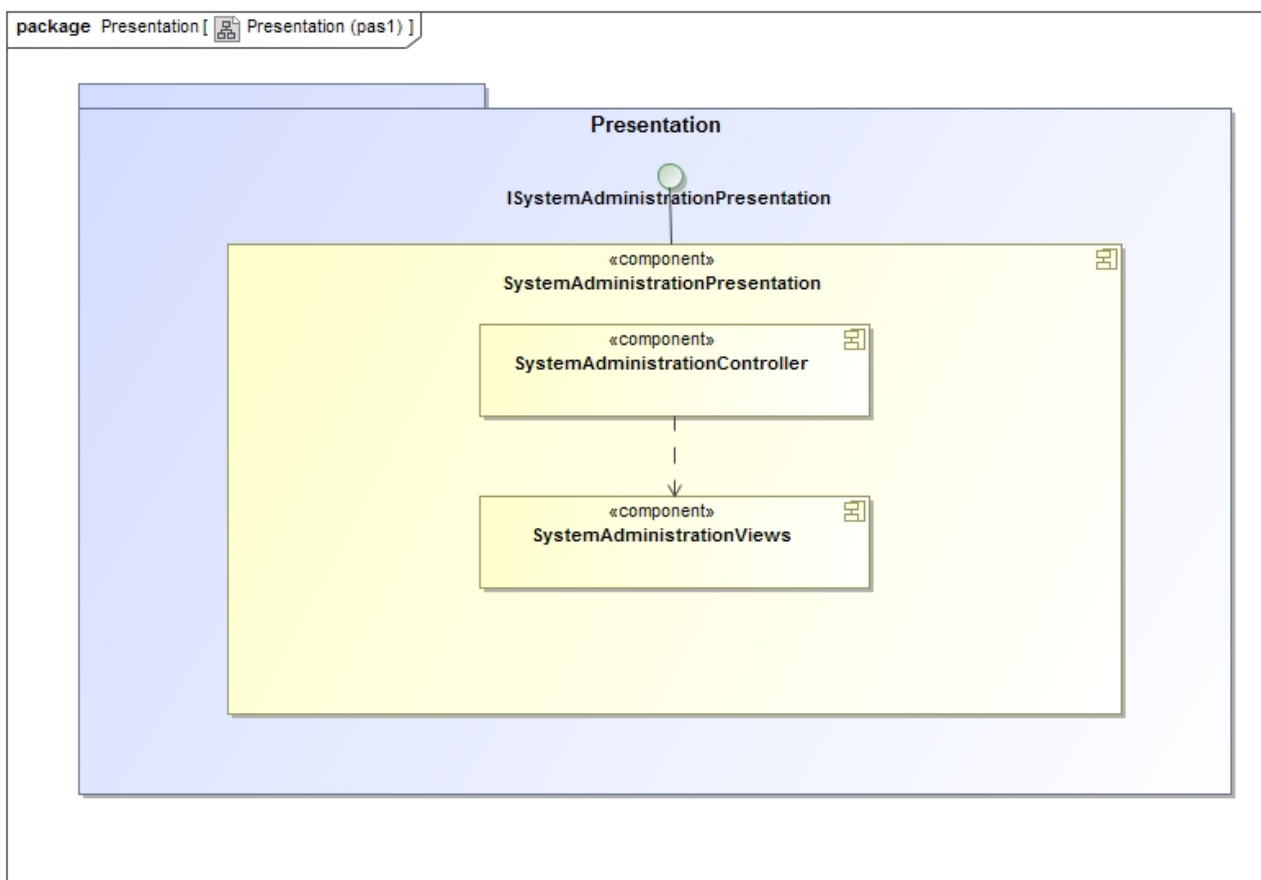
- Refinamiento de los componentes
- Diseño del componente y aplicación del perfil Java EE.

Capa de presentación

Tal como se pedía en el enunciado, se tienen que ofrecer las siguientes operaciones:

- Consultar todos los tipos de especialidad médica
- Añadir un tipo de actividad médica
- Borrar un tipo de actividad médica
- Actualizar la información de un tipo de actividad médica

Si suponemos que utilizamos el patrón MVC (Modelo-Vista-Controlador) y que el modelo lo implementaremos a la capa de negocio, obtenemos un primer refinamiento del componente de presentación separando el componente que hará de controlador y los que harán de vistas.

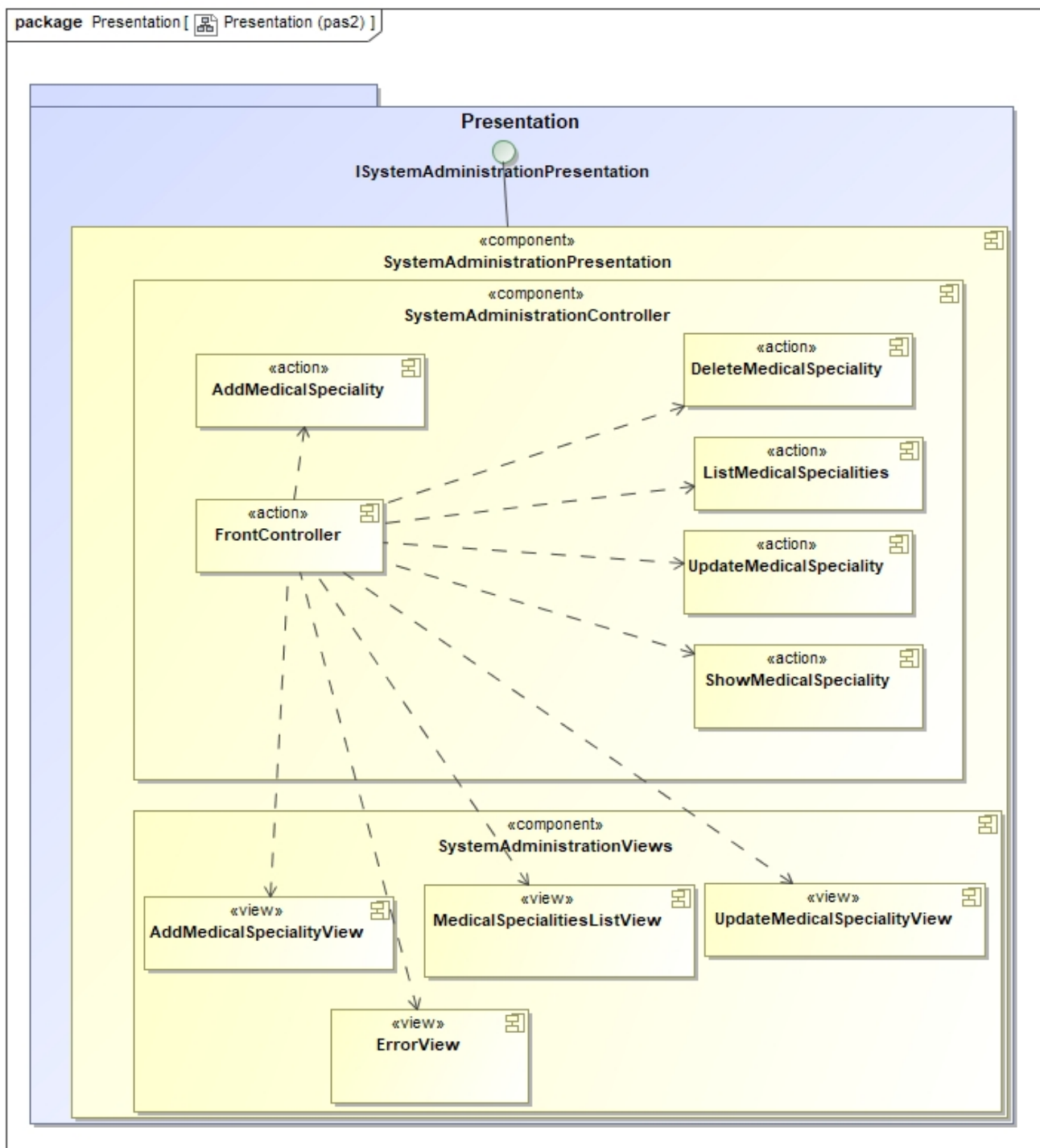


A partir del refinamiento anterior tomamos las siguientes decisiones de diseño:

- Habrá un único controlador para todas las operaciones de un componente (patrón FrontController). Las operaciones no se implementarán dentro del controlador, se utilizará un esquema basado en el patrón Command. El controlador será el responsable de coordinar todo el proceso, pero la implementación está desacoplada. Cada acción que el usuario puede

realizar por medio de la interfaz de usuario (GUI) se mapeará con una relación 1:1 con una vista y el controlador simplemente ejecuta estas acciones.

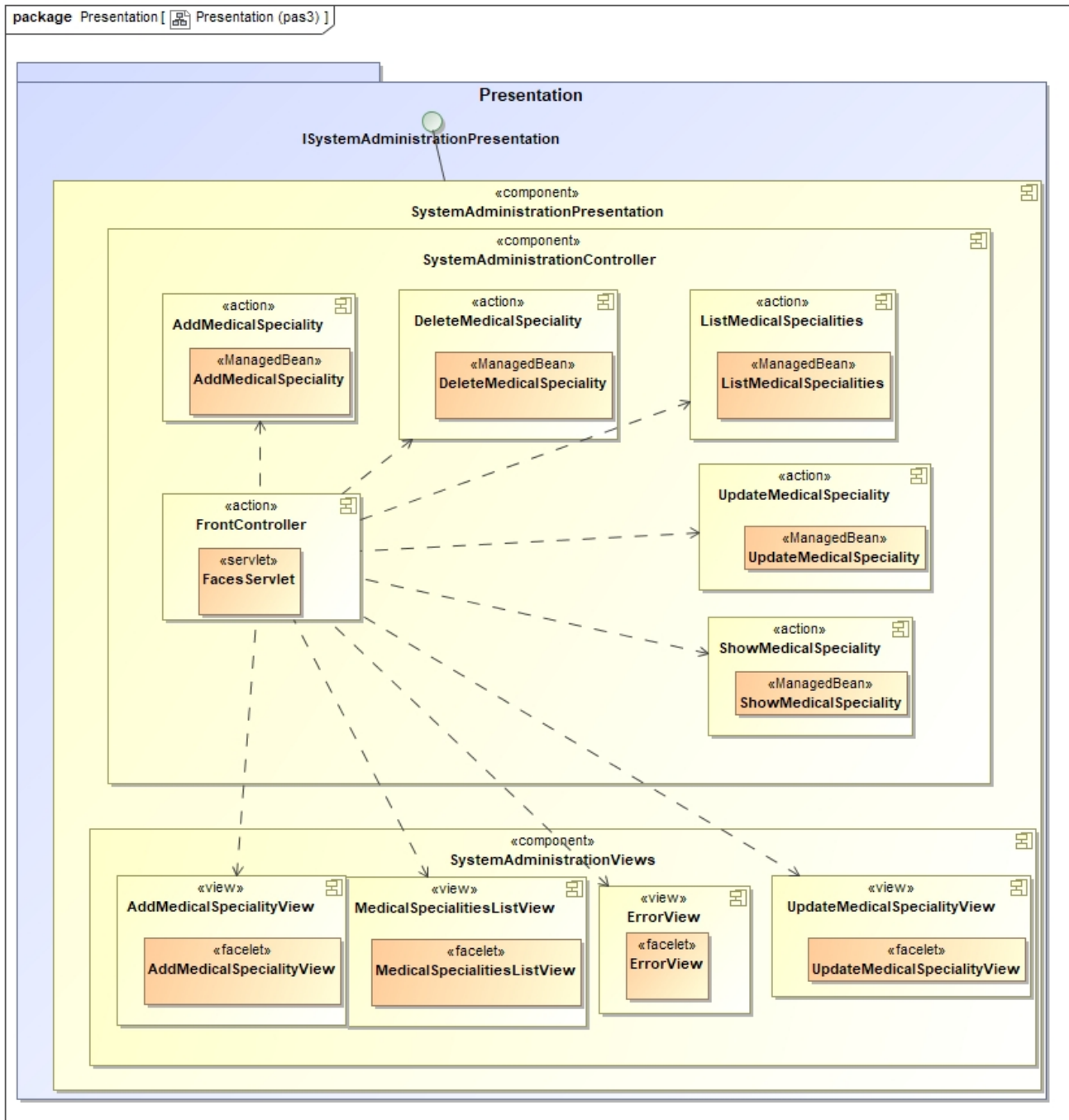
- Se definen las siguientes vistas y acciones:
 - Vistas
 - MedicalSpecialiesListView: Vista inicial que ofrece un listado de todos los tipos de especialidad médica dadas de alta. Da acceso a la operación que permite borrar y a los formularios de alta y modificación.
 - AddMedicalSpecialityView: Muestra los campos que hay que rellenar para dar de alta un nuevo tipo de especialidad médica.
 - UpdateMedicalSpecialityView: Muestra los campos de un tipo de especialidad médica en modo edición para que se pueda modificar su contenido. Para mostrar esta vista nos damos cuenta de que nos hace falta una operación para obtener los datos de un tipo de actividad. Denominaremos a esta operación **getMedicalSpeciality** y se deberá definir tanto en la capa de negocio como en la capa de integración.
 - ErrorView: Muestra los errores que se puedan producir.
 - Acciones
 - ListMedicalSpecialies: Obtiene el listado de todos los tipos de especialidades médicas dadas de alta
 - DeleteMedicalSpeciality: Borra el tipo de especialidad médica
 - AddMedicalSpeciality: Da de alta un nuevo tipo de especialidad médica
 - ShowMedicalSpeciality: Consulta la información de un tipo de especialidad médica
 - UpdateMedicalSpeciality: Modifica la información de un tipo de especialidad médica
- Cualquier operación que produzca un error va a una vista genérica de error. Otra opción hubiera sido presentar los errores sobre las mismas vistas y de esta forma poder ver con qué contenido se relacionan.
- Es igualmente correcto definir una única vista para las operaciones de alta y modificación de tipos de especialidades médica puesto que los campos serán los mismos. Con todo esto obtenemos el siguiente diseño para la capa de presentación (¡fijaos que en este punto todavía no tenemos un diseño dependiente de la tecnología!):



Ahora, tal y cómo especifica el enunciado, hace falta que apliquemos el perfil Java EE a nuestro diseño, para hacerlo tomamos las siguientes decisiones de diseño:

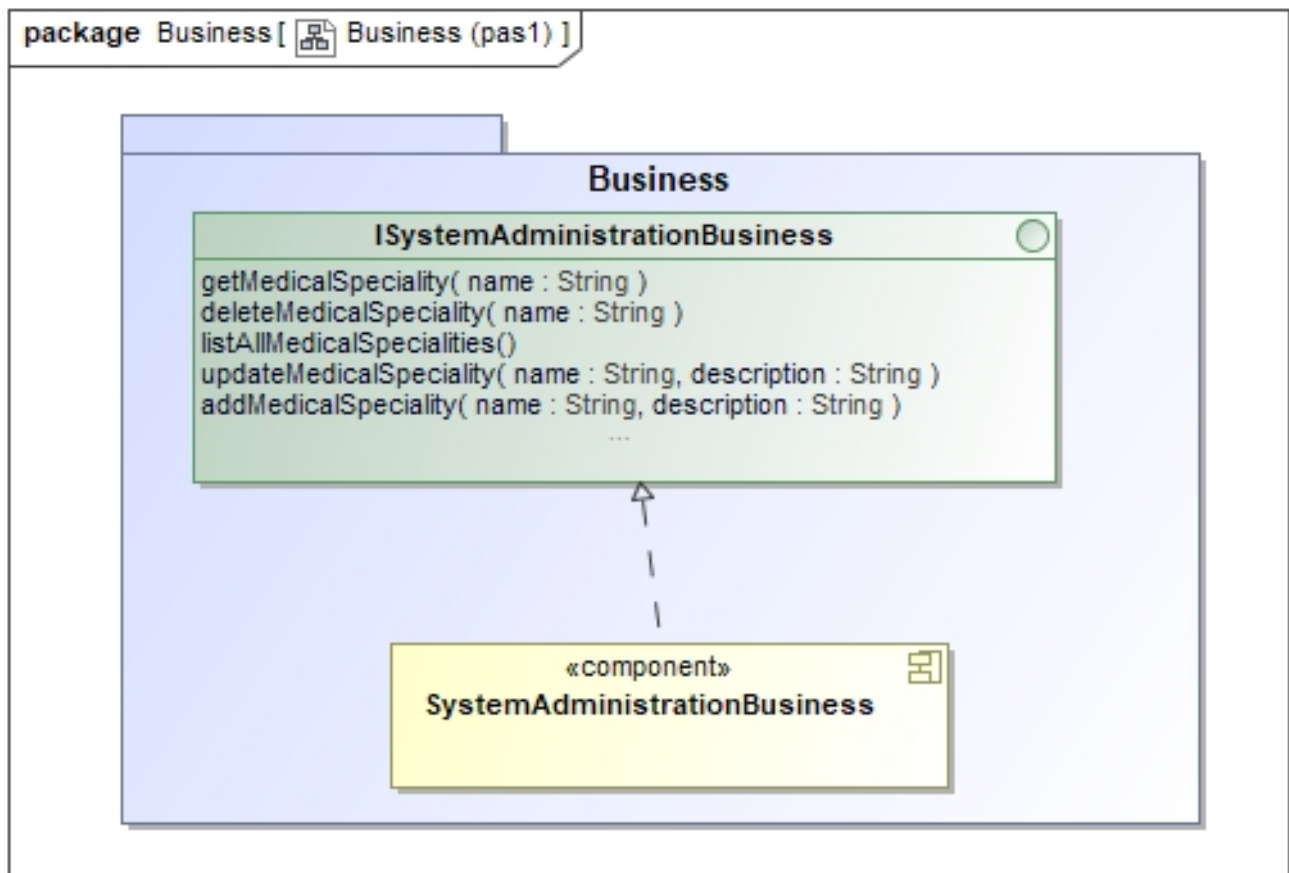
- Para la capa de presentación utilizaremos el framework Java Server Faces (JSF) que viene con la especificación de Java EE. Hay otras muchas opciones igualmente válidas para diseñar la capa de presentación, la opción que se muestra en esta solución es sólo una de ellas.
- El controlador no lo tendremos que implementar, será el servlet Faces Servlet que ya tenemos disponible si utilizamos JSF.
- Las acciones que corresponden a los Commands estarán definidas con ManagedBeans.
- Las vistas las implementaremos con Facelets.
- La vista inicial tendrá un listado con todos los tipos de actividad y permitirá borrarlas, añadir nuevas y actualizar la información.

Implementaremos las vistas con Facelets, el controlador lo proporciona JSF con el servlet Faces Servlet y las acciones serán ManagedBeans. Si aplicamos el perfil Java EE que tenemos en los materiales, obtenemos el siguiente diagrama para los componentes de la capa de presentación (fijaos que en este punto sí que ya tenemos un diseño dependiente de la tecnología, en nuestro caso Java EE):



Capa de negocio

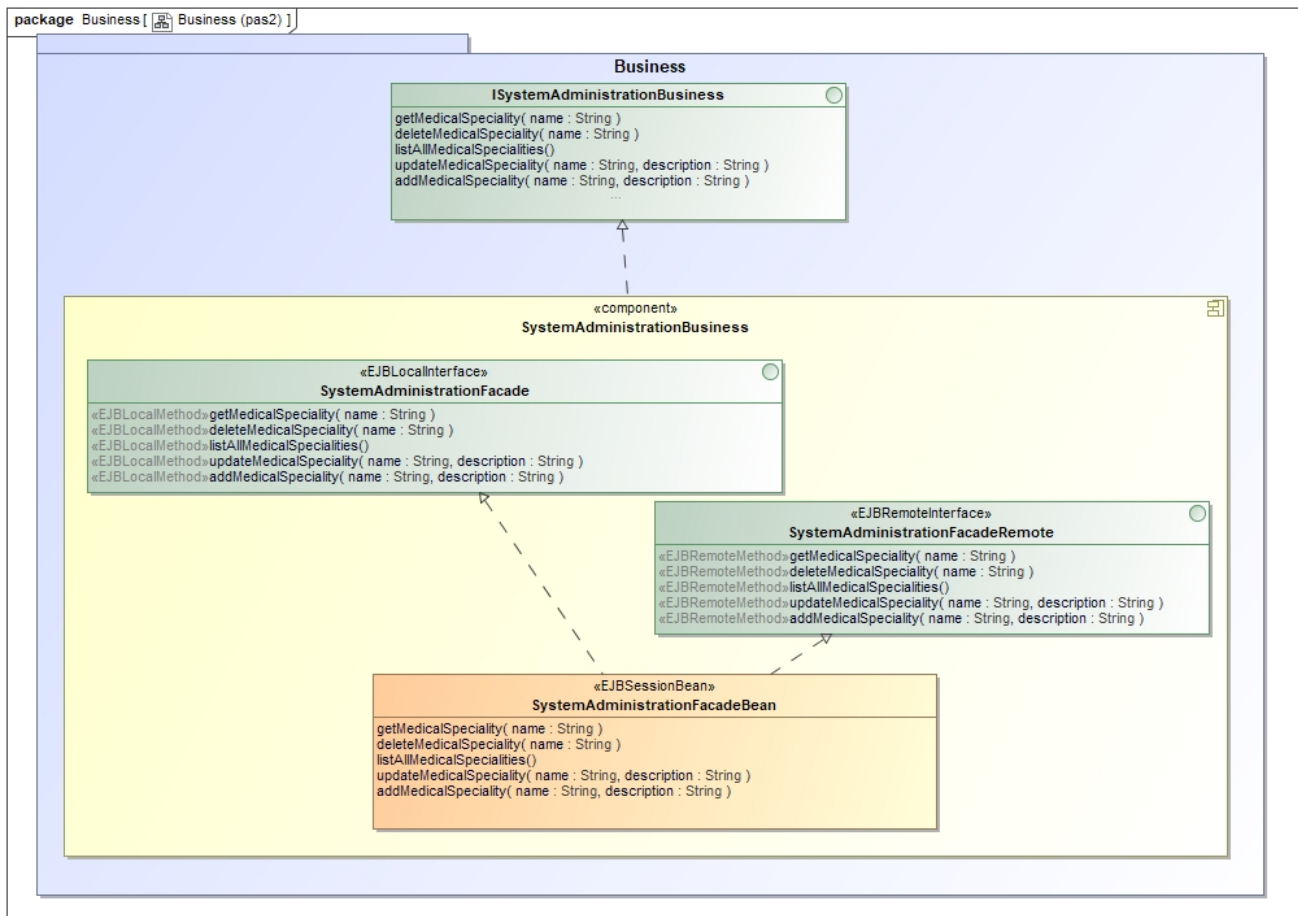
Los componentes de negocio especificado ya están bastante refinados para hacer el diseño y aplicarle el perfil Java EE. Suponemos que implementamos la capa de negocio siguiendo un patrón Fachada.



Ahora hay que aplicar el perfil Java EE a los componentes. Necesitamos dar tanto acceso remoto como acceso local a los componentes de negocio. Las operaciones que hay que realizar son síncronas, no tienen estado y, aunque el enunciado no lo especificaba, decidimos que sea el contenedor Java EE quién gestione las transacciones.

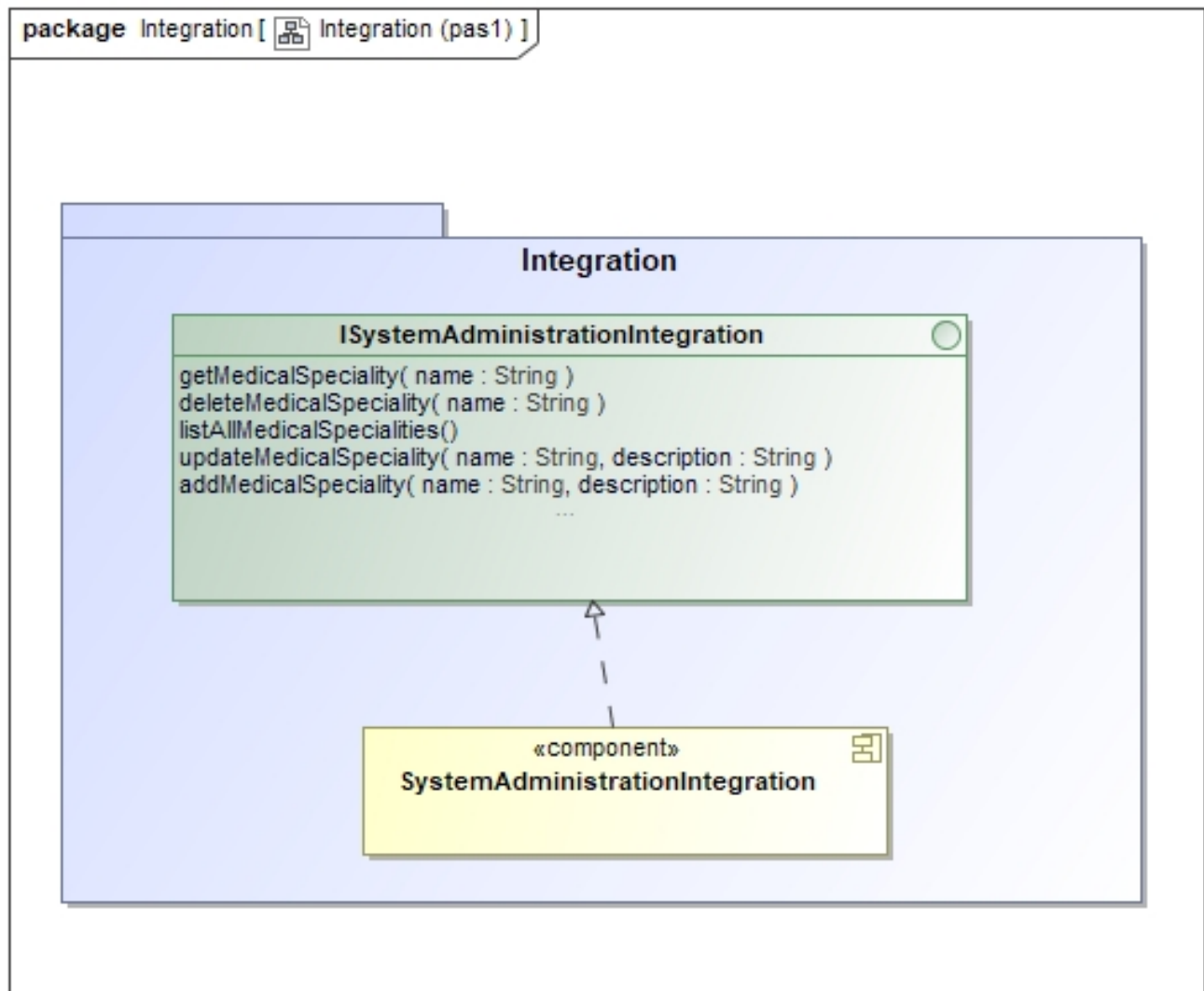
Fijaos que, para implementar la funcionalidad de modificación de los datos de un tipo de actividad, hay que añadir una operación para obtener los datos. Con esto decidimos que la mejor opción para implementar el componente de lógica de negocio es un EJB de sesión sin estado porque el contenedor Java EE nos resuelve toda la complejidad inherente a las comunicaciones remotas, comportamiento transaccional y la gestión de la seguridad.

Si aplicamos el perfil Java EE con las premisas anteriores al componente de negocio obtenemos el siguiente diagrama:

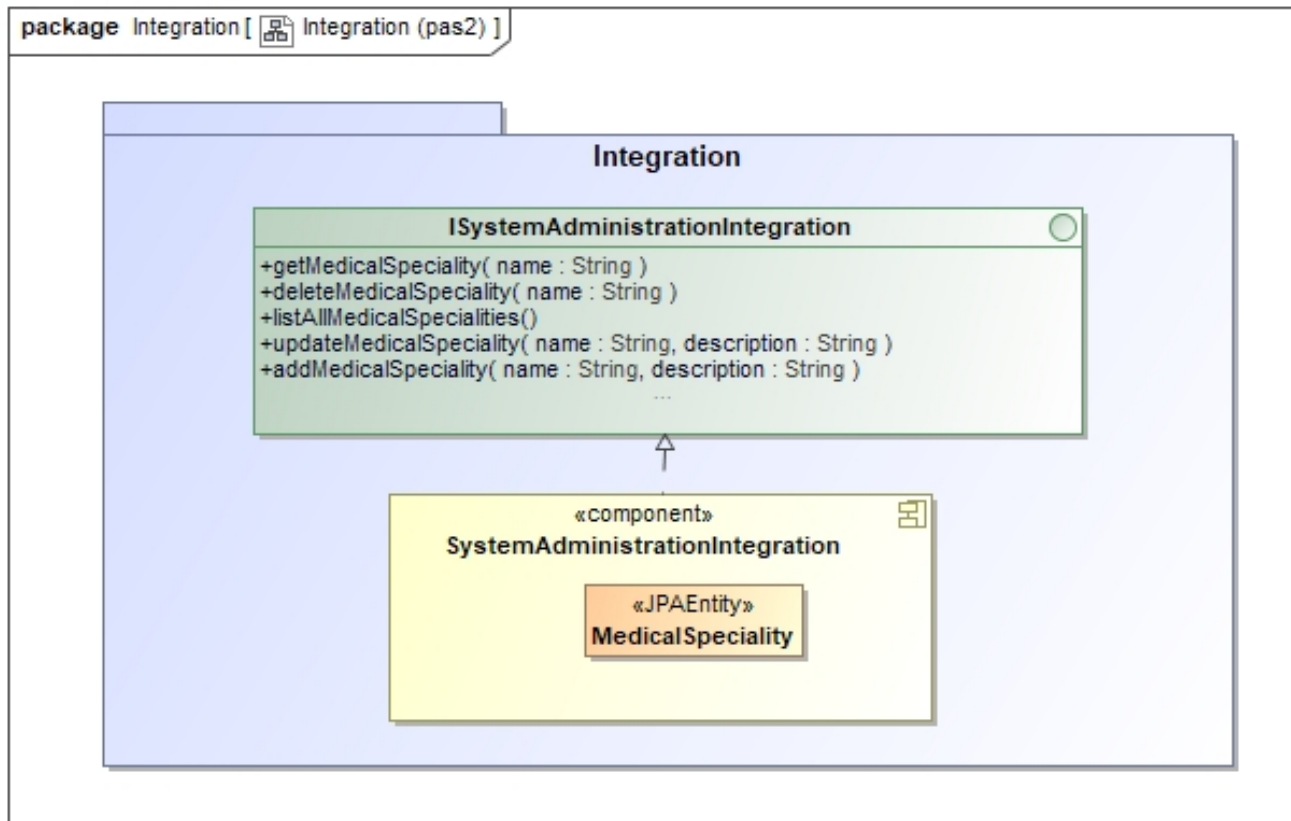


Capa de integración

La capa de integración está formada por un componente `MedicalSpeciality`. Si se refina la parte de integración para representar los componentes obtenemos el siguiente diagrama:



El enunciado nos dice que los datos se almacenarán en una base de datos relacional y que el acceso a la capa de integración desde la capa de negocio será local. Para esta solución optaremos para implementar los componentes con JPA. Fijaos que ésta es una de las posibles decisiones de diseño que podéis tomar en base a las restricciones del enunciado, hay otras opciones igualmente válidas. Aplicando el perfil Java EE y teniendo en cuenta las decisiones de diseño y las consideraciones anteriores obtenemos el siguiente diagrama por la capa de integración:



Ejercicio 2 (1 punto)

Este ejercicio se centra en la instalación y prueba del entorno necesario para poder implementar posteriormente el diseño que se ha obtenido en el ejercicio 1, que se pedirá en la práctica 2. La instalación de todo el software, su configuración, la ejecución de los programas ejemplo y la comprensión y su funcionamiento, son premisas necesarias para poder realizar la práctica 2. Recordad que en el aula de Laboratorio se encuentran todos los tutoriales y casos prácticos necesarios para facilitar esta instalación.

Lo que hay que hacer en este ejercicio y lo que se ha de entregar, se detalla a continuación.

Qué hay que hacer:

- Instalación de PostgreSQL (si no se tiene), JBoss y el Eclipse, (y el JDK, si no se tiene), con la ayuda del tutorial del Laboratorio.
- Desarrollo y ejecución de los ejemplos que hay en el tutorial:
 - Ejemplo con RMI
 - Ejemplo con EJB Session
 - Ejemplo JPA

- Ejemplo Relaciones entre JPA
- Ejemplo Conexión entre Servlets / JSP / JSF y EJB
- Comprensión del funcionamiento de los ejemplos.

Qué hay que entregar:

- Una captura de pantalla para cada uno de los ejemplos con la ejecución, correspondientes a los ejemplos:
- Un ensayo de una página explicando cómo ha sido la experiencia, si se ha encontrado algún problema, cómo se ha resuelto, y cuál ha sido la impresión al leer y ejecutar los ejemplos.

NOTAS:

- Los archivos de captura de pantalla del ejercicio 2 deben estar en formato. JPG o. GIF.
- Todos los archivos deben estar comprimidos en un archivo ZIP, de modo que al descomprimirlo se pueda navegar fácilmente.
- Es necesario que las pantallas que se capturen en el ejercicio 2 sean representativas de la ejecución de los ejemplos; que se vea si se está ejecutando los ejemplos desde dentro o fuera del eclipse, etc.

Solución

El ejercicio 2 sólo se trataba de seguir el tutorial de instalación que tenéis en el Laboratorio para que verifiquéis que vuestra instalación es correcta. No se adjuntan las capturas de pantalla en esta solución.

Ejercicio 3 (2 puntos)

El caso práctico que se presenta con el material de la asignatura se ha hecho el análisis, el diseño y la implementación del componente "Catalog" de la **tienda de mascotas**. En este ejercicio se pedirá estudiar con detenimiento el caso práctico y que se realice una pequeña modificación en este componente.

Qué hay que hacer:

1. Seguir los pasos descritos en el caso práctico para instalar y desplegar la aplicación del caso práctico en Java EE en el servidor de aplicaciones y presentar una imagen de la pantalla principal con listado de categorías y otra con el detalle de un animal.
2. Mostrar en un diagrama de componentes software con el perfil Java EE aplicado al componente "Catalog".
3. Indicar brevemente (máximo 100 palabras) qué deberíamos hacer para ofrecer a los administradores la opción de **añadir categorías del catálogo**. **NO** se pide implementar esta operación.
4. Añadir un nuevo atributo asociado a las categorías del catálogo con una descripción y mostrar esta descripción cuando se consulta el detalle de un animal. Esta pregunta **REQUIERE**

modificar el código que se entrega (desde presentación a integración) y verificar su funcionamiento. Se debe enviar un fichero comprimido del proyecto modificado junto con una imagen de la pantalla que muestra el detalle de un animal con el nuevo atributo y unas indicaciones de las modificaciones realizadas. El archivo comprimido del proyecto que se envíe debe contener el resultado de hacer una exportación del proyecto desde la IDE Eclipse (opción Export -> File System).

Qué hay que entregar:

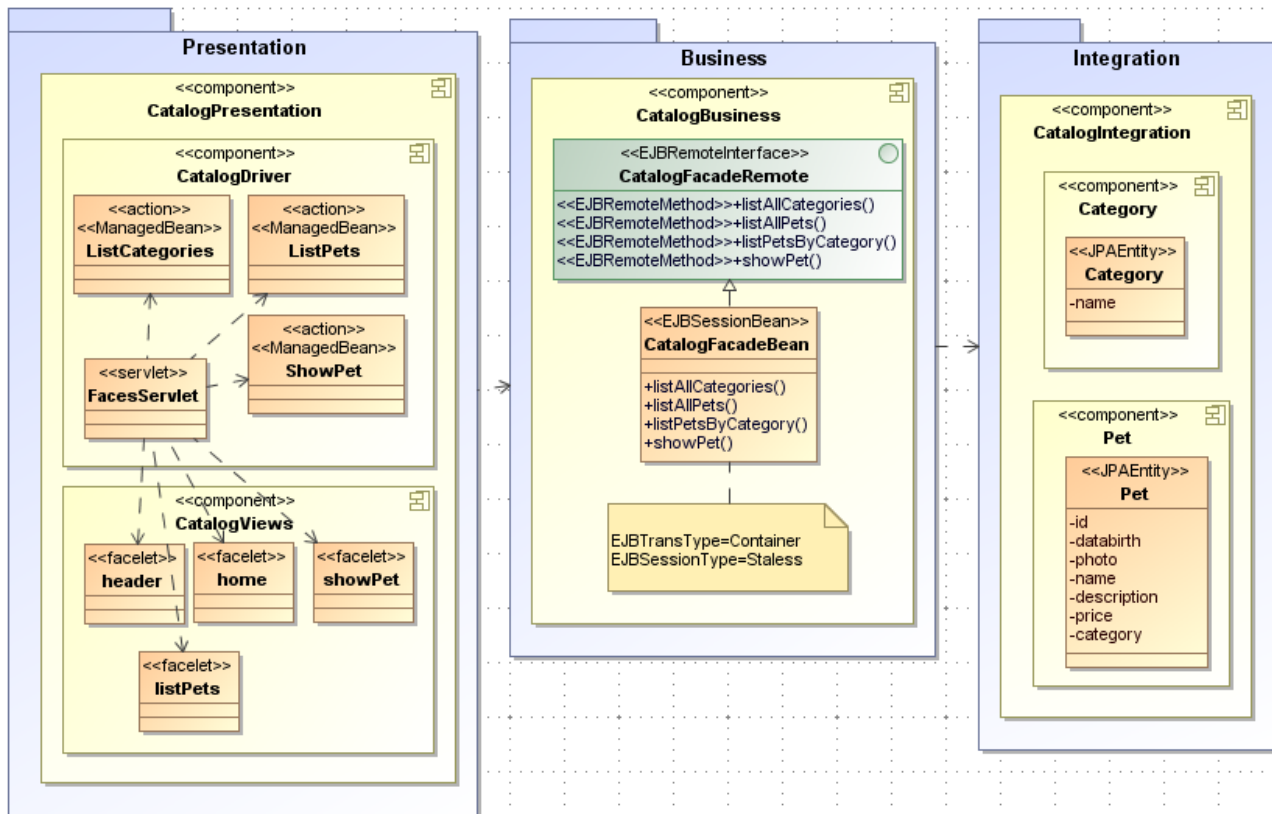
- Imágenes de las pantallas de la lista de categorías y del detalle de un animal obtenidas en el apartado 3.1
- Imagen con el diagrama de componentes del apartado 3.2
- Explicación del apartado 3.3
- Indicaciones de las modificaciones realizadas por el apartado 3.4
- El fichero comprimido con el código modificado en el apartado 3.4

Solución Ejercicio 3 apartado 1)

En el apartado a) del tercer ejercicio teníais que seguir los pasos descritos para instalar y desplegar la aplicación del caso práctico en Java EE en el servidor de aplicaciones y mostrar una imagen de la pantalla principal con listado de categorías y otra con el detalle de un animal. No se adjuntan las capturas de pantalla en esta solución.

Solución Ejercicio 3 apartado 2)

Un posible diagrama de componentes software con el perfil Java EE aplicado del componente “Catalog” siguiendo las decisiones de diseño que tenéis en el caso práctico es el siguiente:



Solución Ejercicio 3 apartado 3)

A grandes rasgos lo que se debería hacer para ofrecer una opción para los administradores que les permita añadir categorías en el catálogo serían:

- Presentación:
 - Añadir una pantalla que permita introducir el nombre de la nueva categoría. Esta pantalla será accesible desde el listado de categorías
 - Crear una acción `AddCategory` para añadir la categoría en el catálogo
 - Utilizar esta opción para añadir la categoría en el catálogo. Esta acción se invocará desde la pantalla que permita introducir el nombre de la nueva categoría.
- Negocio:
 - Añadir una nueva operación remota `addCategory(category: Category)` en el EJB que hace de fachada: **CatalogFacadeBean**.
- Integración:
 - Añadir la operación de alta en la capa de integración o utilizar la que proporciona JPA.

Solución Ejercicio 3 apartado 4)

Los cambios que había que hacer para añadir una descripción a las categorías y mostrarla cuando se muestra el detalle son los siguientes:

- Añadir el campo de descripción en la tabla de categorías de la base de datos.
- Añadir el campo a la entidad JPA que representa a las categorías y mapearlo en la columna de la tabla que contiene la información (en caso de no utilizar la nomenclatura por defecto).
- En la capa de negocio no hay que hacer nada porque ya tenemos un método que nos devuelve el listado de categorías.
- En la capa de presentación hay que modificar el facet que muestra el detalle del animal para mostrar también la descripción de la categoría a la que pertenece el animal.
- Opcionalmente se puede modificar también el facet que muestra el listado de animales para mostrar también la descripción de la categoría

Resumen de lo que se debe entregar para toda la Práctica 1:

- Todos los archivos deben estar comprimidos en el archivo ZIP de la entrega (ApellidosNombre_ISCSDPractica1), que contenga lo siguiente:
- Los archivos de captura de pantalla (.jpg) dentro de una carpeta para cada ejercicio: ejercicio1, ejercicio2 y ejercicio3.
- Un documento / memoria que resuelva los ejercicios propuestos (organizado por ejercicios).
- Un fichero comprimido con el proyecto modificado al ejercicio 3, apartado 4.

Recursos

Recursos Básicos

- Módulo 3: Desarrollo de software basado en componentes
- Módulo 6: Java EE
- Caso práctico de estudio: Diseño
- Caso práctico de estudio: Programación (Aula Laboratorio)
- Tutorial instalación (Aula Laboratorio)

Criterios de evaluación previos

- La Práctica debe resolverse de forma **estrictamente individual**. En caso de detectar copias **se penalizará la actividad con una D como nota**.
- El peso de cada ejercicio está indicado en el enunciado.
- Es necesario justificar la respuesta a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que entregar un único documento ZIP con las respuestas a todos los ejercicios. El nombre del archivo debe ser: *ApellidosNombre_ISCSDPractica1.zip*.

Este documento se entregará en el espacio de Entrega y Registro de Evaluación Continuada (REC) del aula antes de las **23:59 horas del día 09 de mayo de 2019**. No se aceptarán entregas fuera de plazo.

